Environment Diagram

Computer Science Mentors 61A

March 6, 2026

1 Environment Diagram

1. Draw the environment diagram that results from running the code.

```
six = [6]
seven = [7]

def six_seven(sixty, seventy):
    def six_or_seven(s):
        if s.pop() == 6:
            return 67
        else:
            return 7
    seven = six_or_seven(sixty)
    sixty.append(seventy.extend(six))
    sixty.append([seven])
    return sixty

ss = six_seven(six, seven)
print(ss)
```

Created by Jim Fang, Bryan Adolfo Ventura Benitez, Rajoshi Basu Worksheet feedback: links.jimfang.dev/feedback

2. You are at a buffet! Yummy.

   You see that the buffet line has many different dishes: p (protein), v (vegetable), r (rice), and f (fruit). You note down the items in line as **options**.

   Now, you are very picky about the order in which food reaches your plate and want to get food **in a fixed order**, but you are also very indecisive and want to know if different orders of getting food is possible.

   Complete the **buffet_possible** function, that takes in the buffet options (options) and your desired choice (choice), and returns whether it is possible to get your choice from the line.

```
def buffet_possible(options, choice):
    """
    >>> buffet_possible("pvprprrpff", "p")
    True
    >>> buffet_possible("pvprprrvpff", "pvpvp")
    True
    >>> buffet_possible("pvprprrvpff", "fff")
    False
    >>> buffet_possible("pvprprrvpff", "")
    True
    """

    if _________:
        return ___________
    elif ___________:
        return ___________
    else:
        if ___________:
            return ___________
        else:
            return ___________
```

3. Finally, you have decided what to eat.

   Now, given **options** and **choice**, you want to know how you should get food from the line.

   Complete the **buffet** function, that takes in the buffet options (options) and what you want (choice), and returns the list containing lists of possible positions to take the dishes.

```
def buffet(options, choice):
    """
    >>> buffet("pvprprrpff", "p")
    [[0], [2], [4], [7]]
    >>> buffet("pvprprrvpff", "rr")
    [[3, 5], [3, 6], [5, 6]]
    >>> buffet("pvprprrvpff", "pvprprrvpff")
    [[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]]
    >>> buffet("rrfff", "rrrr") # Not possible
    []
    >>> buffet("rrfff", "") # Not selecting anything
    [[]]
    """

    def helper(options, choice, current_index):
        if __________:
            return [[]]
        if ___________:
            return []
        with_current = ___________
        without_current = ___________
        if ___________:
            return ___________
        else:
            return ___________

    return ___________
```